

AI 활용 기술 문서

Tiny Legends · Unity 기반 방치형 RPG의 AI 협업 구조, 프롬프트 및 검증 방식

프로젝트	Inflearn_Seed / IdleProject (Tiny Legends)
엔진	Unity 6000.3.13f1 · URP 17.3.0
AI 도구	Claude Code, OpenAI Codex, Unity AI Assistant / Unity MCP
작성 기준일	2026-08-10
개발 형태	1인 개발 · AI 보조 활용

1. AI 활용 개요

본 프로젝트에서 AI는 단순 코드 자동완성이 아니라 저장소를 조사하고, 코드를 수정하고, Unity 에디터에서 결과를 직접 검증하는 개발 에이전트로 사용되었습니다. 개발자는 기획·에셋 선정·플레이 감각·최종 의사결정을 담당하고 AI 결과를 반복 검수했습니다.

Claude Code (Anthropic)	초기·주요 시스템 구현, 리팩터링, 디버깅, 데이터/에디터 도구 생성
OpenAI Codex	후속 기능 수정, 모바일 호환성 감사, Unity MCP 기반 런타임 검증, 제출 문서 생성
Unity AI Assistant / MCP	AI가 Unity 내부 C# 실행, 씬·에셋 조회/수정, 콘솔 로그 및 렌더 상태 검증

사람과 AI의 역할 구분

게임 기획, 재미 판단, 에셋 선정, 실제 플레이 테스트, 결과 승인·반려	C# 구현, 에디터 자동화, 데이터 생성, 오류 추적, 검증 코드 실행, 문서 초안

2. 기술 구조

자연어 지시 → 저장소/기존 구현 조사 → 코드 또는 에디터 자동화 작성 → Unity MCP로 컴파일·실행·콘솔·렌더 결과 확인 → 실패 시 원인 수정 → 개발자 보고 및 플레이 검수의 순환 구조입니다.

Unity MCP 검증 범위

- 에디터 내부 C# 컴파일 및 실행
- 씬 오브젝트·컴포넌트·직렬화 참조 검사
- 런타임 오브젝트 생성과 UI 렌더 상태 확인
- Unity Console 오류·경고 및 스택 추적 회수
- Android 빌드 전 셰이더·Resources·씬 구성 검사

재현 가능한 에디터 자동화

대규모 UI와 데이터는 YAML을 직접 편집하기보다 ShopPanelBuilder, EquipmentPanelBuilder, LocalizationBinder, CharacterAnimatorSkillSetup 등의 Editor 도구를 생성해 다시 실행 가능한 형태로 관리했습니다.

3. AI 활용 구현 사례

영역	AI 활용 내용
전투	자동 타겟·이동·공격·스킬 연계, 웨이브/보스, 피격·드롭 연출 구현 및 오류 수정
데이터	ScriptableObject 기반 캐릭터·스킬·아이템·스테이지 규칙 생성 및 검증
UI	Equipment·Skill·Dungeon·Shop 계층 생성, Safe Area와 다양한 화면비 대응
저장/로그인	Firebase 익명·Google 인증, Firestore 저장 흐름과 실패 분기 보강
로컬라이제이션	한국어·일본어·영어 3개 언어를 단일 CSV로 관리하고 씬 키를 자동 배선
사운드	SfxSynth에서 파형·잡음·필터·포락선을 조합해 효과음 61종을 코드로 절차적 합성
모바일 검증	중복 AudioListener, Missing Script, Safe Area, shader stripping, Resources 경로를 자동 감사

대표적인 검증 사례

- 데미지 텍스트가 생성되지만 보이지 않는 문제를 런타임에서 직접 강제 호출해 Canvas 정렬 문제로 특정
- 저장 조회 예외가 신규 계정으로 오인되어 Select로 이동하는 분기를 성공/없음/실패의 3상태로 분리
- 빌드에서만 Resources.Load가 실패할 프리팹 경로를 실제 Resources 구조로 이동하고 빌드 전 검증기 추가
- 모바일 씬의 중복 AudioListener와 끊어진 Missing Script 슬롯을 전수 감사해 제거

4. 주요 프롬프트 및 지시 사항

상시 규칙

- 기존 구현과 계층을 먼저 조사하고 같은 규칙을 따를 것
- 주석은 동작 설명보다 해당 구현을 선택한 이유를 남길 것
- UI 레이아웃과 기능 동작을 구분하고 불필요한 런타임 스크립트를 늘리지 않을 것
- 모바일 설명 문구는 짧게 유지하고 번역은 단일 CSV에서 관리할 것
- 밸런스 변경은 근거 수치를 제시한 뒤 적용할 것
- 작업 후 Unity 컴파일·콘솔·실제 런타임 상태를 직접 확인할 것

프롬프트 예시 1 · 기존 규칙 확장

“Skill5 애니메이션을 기존처럼 연결하고, Effects/Row 프리팹을 실행해 화면의 모든 몬스터에게 5차례 피해를 주게 해줘.”

핵심은 ‘기존처럼’이라는 제약입니다. AI가 기존 스킬의 Animator 파라미터, Animation Event, 데이터 구조를 먼저 조사한 뒤 동일한 패턴으로 확장하도록 했습니다.

프롬프트 예시 2 · 증상 기반 디버깅

“쿨타임이 남아 있는데 T-Pose가 나오는 원인을 찾아서 절대 나오지 않게 해줘.”

원인을 미리 단정하지 않고 증상과 기대 결과를 제공해 AI가 코드·Animator·에셋 참조를 함께 조사하도록 했습니다.

프롬프트 예시 3 · 모바일 호환성

“게임 자체의 변경점이 없는 범위에서 모바일 실행 시 문제가 될 부분을 직접 파악하고 수정해.”

게임 규칙과 연출은 유지하고 Android 설정, 씬 중복, Safe Area, Resources, 웨이더, 입력과 비동기 저장을 감사하도록 범위를 정의했습니다.

5. 외부 에셋 및 오픈소스

엔진·SDK·오픈소스

이름	용도	라이선스/출처
Unity / URP / TMP / Input System	엔진·렌더링·UI·입력	Unity Terms 및 공식 패키지 라이선스
Firebase Unity SDK 13.14.0	Auth·Firestore	Apache License 2.0 · github.com/firebase/firebase-unity-sdk
Google Sign-In for Unity	Google 인증	Apache License 2.0
External Dependency Manager	Android 의존성	Apache License 2.0
Noto Sans KR / JP	한국어·일본어 폰트	SIL Open Font License 1.1 · github.com/notofonts/noto-cjk
Mixamo animations	플레이어 애니메이션	Adobe ID 기반 개인·상업·비영리 프로젝트 로열티 프리 사용 안내

Unity Asset Store 에셋

에셋	제작자	사용처
Suntail Village	Raygeas	전투 배경 환경
Mummy Monster	amusedART	몬스터 모델·애니메이션
Modern UI Pack v5.5.28	Michsky	UI 컴포넌트·애니메이션
Casual RPG / Environment VFX	Lana Studio	스킬·환경 이펙트
Hyper Casual FX Vol.1-2	Kyeoms (VFX_Klaus)	타격·획득 연출
Top Down Effects	SineVFX	스킬 범위 연출
Wooden Shield Pack	Korzanowski	방패 모델

위 유료·무료 에셋은 Unity Asset Store를 통해 취득했으며 Unity Asset Store EULA를 따릅니다. 에셋 원본을 독립적으로 재배포하지 않고 게임 빌드의 구성 요소로만 사용합니다.

사운드

효과음 61종	외부 음원 없이 프로젝트 코드 SfxSynth가 런타임 생성
Title_BGM.wav / Main_BGM.wav	프로젝트에는 파일만 있고 원작자·취득 경로·라이선스 기록이 없음 — 제출 전 반드시 확인

저작권 제출 전 필수 확인

BGM 2곡의 원작자, 곡명, 취득 URL 또는 구매 내역, 허용 라이선스를 확인해 이 문서의 사운드 표를 교체해야 합니다. 확인할 수 없다면 해당 BGM을 사용 권한이 명확한 음원으로 교체해야 합니다.

공식 참고 링크

- Unity Asset Store Terms: <https://unity.com/legal/as-terms>
- Firebase Unity SDK: <https://github.com/firebase/firebase-unity-sdk>
- Noto CJK: <https://github.com/notofonts/noto-cjk>
- Adobe Mixamo FAQ: <https://helpx.adobe.com/creative-cloud/faq/mixamo-faq.html>
- 프로젝트 저장소: https://github.com/PinousCEO/Inflearn_Seed

6. 한계와 책임

AI가 생성한 코드는 개발자가 검토하고 Unity 에디터 및 실제 플레이로 확인했습니다. AI 결과는 오류 가능성이 있으므로 최종 제출·배포 책임은 개발자에게 있습니다. 외부 저작물의 라이선스는 취득 당시 계정과 구매 내역을 기준으로 최종 재확인해야 합니다.

1인 개발 프로젝트이므로 별도의 팀원 룰 기술서는 제출하지 않습니다.